Исследование методов проверки микросервисных архитектур, проектируемых большими языковыми моделями

Дичковский Владимир Андреевич

Аннотация

Автоматическое проектирование микросервисных архитектур с помощью больших языковых моделей позволяет получать варианты топологии из текстовых требований, однако синтаксически корректное представление не доказывает выполнения требований к производительности. В работе формализуется задача проверки таких архитектур и предлагается методика сравнительного исследования трёх уровней валидации: структурных ограничений графа, аналитических моделей теории массового обслуживания и дискретно-событийного моделирования. Рассматриваются условия применимости моделей очередей, конечные буферы, случайная нагрузка и статистическая неопределённость перцентилей задержки. Предложен план экспериментов для сопоставления одношаговой генерации и итеративного исправления по результатам проверок, в том числе для локальных языковых моделей. Определены эталоны, измеряемые показатели и критерии достоверности. Количественная проверка гипотез запланирована, поэтому результаты и утверждения о превосходстве методов пока не приводятся.

Ключевые слова: микросервисная архитектура, большие языковые модели, валидация архитектуры, теория массового обслуживания, дискретно-событийное моделирование, нефункциональные требования, экспериментальное исследование.

1 Введение

Микросервисная архитектура состоит из взаимодействующих компонентов с ограниченными вычислительными ресурсами. Наличие корректной схемы связей не означает, что система будет обслуживать запросы с требуемой задержкой и без перегрузки. Структурную корректность и поведение под нагрузкой необходимо проверять отдельно [1, 2].

Большие языковые модели (Large Language Models, LLM) применяются для преобразования текстовых требований в архитектурные представления [3,4]. Однако их собственная оценка пропускной способности предложенной топологии не заменяет расчёт. Нефункциональные требования (Non-Functional Requirements, NFR) включают ограничения на задержку, пропускную способность, стоимость и доступность.

Здесь сравниваются три уровня проверки: ограничения графа, аналитические модели теории массового обслуживания (TMO) и дискретно-событийное моделирование (Discrete-Event Simulation, DES) запросов и ресурсов с помощью SimPy [5]. Заключение о выполнении ограничения может быть получено только в границах явно заданной модели и статистической неопределённости.

1.1 Актуальность автоматизированного архитектурного проектирования

Микросервисная архитектура допускает множество вариантов разбиения системы на сервисы, маршрутизации запросов, репликации и распределения ресурсов. Даже функционально эквивалентные варианты различаются по задержке, пропускной способности и стоимости. Ошибки проектирования, обнаруженные после реализации, требуют пересмотра взаимодействий компонентов [1,6].

Большие языковые модели позволяют получать архитектурные варианты непосредственно из текстовых требований, однако не гарантируют выполнения количественных ограничений. Современные исследования подтверждают интерес к применению генеративного ИИ в архитектурном проектировании, одновременно отмечая необходимость независимой оценки качества решений [7,8]. Поэтому исследуется сочетание генерации вариантов с детерминированной структурной проверкой и моделированием производительности. Перспективность подхода не означает доказанной замены архитектора: выбор модели нагрузки, допустимых компромиссов и окончательного решения остаётся ответственностью специалиста.

2 Цель и задачи исследования

Цель — разработать и экспериментально оценить метод проверки микросервисных архитектур, создаваемых большими языковыми моделями, по структурным и количественным нефункциональным ограничениям.

Поставлены следующие задачи:

1. Формализовать граф архитектуры, профиль нагрузки, проверяемые ограничения и границы применимости оценочных моделей.
2. Разделить структурные, аналитические и имитационные проверки, включая явное указание случаев, когда заключение получить нельзя.
3. Сформировать набор контрольных сценариев с различными нагрузками, очередями и маршрутизацией, включая заведомо ошибочные архитектуры.
4. Сравнить одношаговую генерацию и итеративное исправление с разными видами обратной связи на одинаковых сценариях и моделях LLM.
5. Измерить риск ложного подтверждения соответствия, точность численных оценок, долю неопределённых случаев и вычислительные затраты.

Для последующего экспериментального исследования сформулированы следующие гипотезы:

1. Диагностическая обратная связь при заданном пределе числа итераций повышает долю структурно и функционально допустимых архитектур по сравнению с однократной генерацией.
2. Аналитические модели очередей позволяют обнаруживать перегрузку с небольшими вычислительными затратами при соблюдении их предпосылок, но не заменяют проверку перцентильных ограничений для произвольного профиля нагрузки.

3. Дискретно-событийное моделирование позволяет выявить дополнительные нарушения количественных требований в сценариях с конечными очередями, нестационарными потоками и сложной маршрутизацией, но увеличивает время оценки и само зависит от корректности предположений.

Эти утверждения являются *проверяемыми предположениями*, а не полученными результатами. Их необходимо сопоставить с независимыми эталонами и статистической неопределённостью.

3 Предшествующие исследования

Существующие исследования решают две связанные, но различающиеся задачи: *построение архитектурных представлений из требований* и *прогнозирование характеристик уже заданной архитектуры*. Их сопоставление позволяет уточнить предмет исследования, не приписывая предложенному подходу новизну отдельных известных методов.

Большие языковые модели (Large Language Models, LLM; большие языковые модели) применяются для генерации UML-диаграмм и архитектурных представлений из естественно-языковых требований [3, 9]. В частности, NOMAD использует несколько агентов, промежуточное структурированное представление и проверку UML-модели [10]. MAAD и C4Modeler рассматривают многоагентное проектирование архитектур и проверку получаемых представлений [11,12]. Таким образом, ни использование LLM, ни промежуточная модель, ни структурная валидация сами по себе не составляют новизну рассматриваемой работы.

Для оценки производительности распределённых систем уже разработаны компонентные модели Palladio [6], имитационные инструменты PerfSim и CloudNativeSim [13, 14] и экспериментальные микросервисные стенды DeathStarBench [15]. Аналитическую основу оценки загрузки и ожидания составляют модели теории массового обслуживания [2, 16]. Эти средства позволяют исследовать заданную архитектуру, но сами по себе не определяют способ изменения генерируемой LLM топологии на основании выявленных нарушений требований.

Исследовательский пробел состоит не в отсутствии средств генерации или моделирования, а в недостаточной изученности их *совместного применения* для проверяемой корректировки микросервисной архитектуры: требуется установить, насколько разные уровни независимой проверки изменяют долю допустимых решений, риск ложного подтверждения нефункциональных требований и вычислительные затраты. Именно эти величины, а не число создаваемых диаграмм, рассматриваются далее.

4 Постановка задачи

Входной сценарий задаётся кортежем

$$\mathcal{S} = \langle \mathcal{D}, \mathcal{F}, \mathcal{W}, \mathcal{R} \rangle, \tag{1}$$

где $\mathcal{D}$ — текстовое описание предметной области, $\mathcal{F}$ — функциональные сценарии, $\mathcal{W}$ — профиль нагрузки, $\mathcal{R}$ — ограничения на количественные показатели. Интенсивность λ измеряется в запросах в секунду (requests per second, RPS).

Синтезируемая архитектура представлена ориентированным графом $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Вершина $v \in \mathcal{V}$ содержит тип компонента, число реплик r_v, число независимых каналов обслуживания c_v, предел длины очереди q_v и распределение времени обработки. Ребро $e \in \mathcal{E}$ определяет направление и тип взаимодействия, маршрутизацию и сетевую задержку. В

частности, максимальная конкурентность *не равна* автоматически числу независимых каналов обслуживания.

Структурный предикат $V_{\text{str}}(\mathcal{G})$ отражает соблюдение правил связности, типов и разрешённых переходов. Для каждого количественного требования оценщик должен формировать один из четырёх выводов: *выполнено*, *нарушено*, *недостаточно оснований* или *не оценивалось*. Последний вывод нужен, если модель не содержит параметров для расчёта требуемого показателя.

Для ограничения $L_p \le L_{\max}$ на p-й перцентиль клиентской задержки имитация должна сообщать оценку $\widehat{L}_p$ и доверительный интервал $[L_p^-, L_p^+]$. Если $L_p^+ \le L_{\max}$, допускается заключение о выполнении требования *в принятой модели и на заданном уровне доверия*. Если $L_p^- > L_{\max}$, фиксируется нарушение. При пересечении интервала с порогом результат остаётся статистически неопределённым. Нельзя делать вывод по единственному случайному прогону или заменять перцентиль средним временем ответа.

Минимизация инфраструктурных затрат $C(\mathcal{G})$ при удовлетворении ограничений является дополнительной поисковой задачей. Итеративная генерация может получить допустимый кандидат, но без доказательства оптимальности не имеет оснований объявлять найденное решение глобальным минимумом.

5 Метод проверки и итеративной корректировки

Первоначально языковая модель преобразует сценарий $\mathcal{S}$ в кандидат $\mathcal{G}_0$. Проверка возвращает структурированные причины ошибок $\mathcal{F}_k$. По ним модель может предложить исправленный кандидат:

$$\mathcal{G}_{k+1} = \text{LLM}(\mathcal{S}, \mathcal{G}_k, \mathcal{F}_k), \qquad 0 \le k < K, \tag{2}$$

где K — предел числа итераций. Эта запись описывает алгоритм, но *не доказывает его сходимость*. Цикл завершается после успешных проверок, исчерпания ограничений или обнаружения неподдерживаемых исходных условий.

5.1 Структурные проверки и диагностическая обратная связь

На первом уровне проверяются существование узлов, допустимость связей, достижимость необходимых компонентов и отсутствие запрещённых циклов. Ошибки форматирования также фиксируются, но не смешиваются с содержательными нарушениями. Формат JSON (JavaScript Object Notation; текстовое представление структурированных данных) облегчает передачу графа между этапами, но сам по себе не гарантирует его смысловую корректность.

Модель архитектурных представлений C4 (Context, Containers, Components, Code; контекст, контейнеры, компоненты, код) применяется для визуализации. Корректная C4-диаграмма не является свидетельством выполнения ограничений по задержке и пропускной способности. Адресное сообщение об ошибке, например «ребро указывает на отсутствующий узел», позволяет проверить эффект целенаправленной коррекции по сравнению с полным повторным генерированием.

5.2 Аналитические и имитационные проверки

После установления структурной допустимости рассчитываются поступающие на узлы потоки и аналитические показатели загрузки. Расчёт допустим только при известной дисциплине обслуживания, числе каналов и распределениях поступлений и обслуживания.

Имитация расширяет класс исследуемых ситуаций: можно воспроизвести конечные очереди, случайные времена обработки, пиковые нагрузки и отказы в приёме запросов.

Каждый уровень возвращает не только значение показателя, но и его предпосылки. Неизвестное число каналов, неподдержанная семантика асинхронного вызова, отсутствие модели отказов или неполный клиентский путь не должны автоматически приводить к заключению «требования выполнены». Подробное сравнение этих уровней составляет предмет планируемого эксперимента.

6 Структурная допустимость архитектурного графа

Проверка корректности архитектуры задаётся совокупностью формальных ограничений, независимых от конкретной языковой модели. Для графа $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ примерами таких ограничений служат:

1. **Уникальность идентификаторов узлов**: разные вершины имеют разные идентификаторы:
$$\forall u, v \in \mathcal{V}, \quad u \neq v \Rightarrow id(u) \neq id(v). \tag{3}$$
2. **Ссылочная целостность**: оба конца каждого ребра принадлежат графу:
$$\forall (u, v) \in \mathcal{E}, \quad u \in \mathcal{V} \wedge v \in \mathcal{V}. \tag{4}$$
3. **Достижимость**: от клиентского входа достижимы узлы, необходимые для выполнения соответствующего функционального сценария.
4. **Отсутствие циклов в выбранной модели синхронных вызовов**: для рассматриваемой аналитической модели предполагается ориентированный ациклический граф (Directed Acyclic Graph, DAG). Это *ограничение модели*, а не утверждение, будто любой цикл между микросервисами неизбежно приводит к взаимоблокировке.

6.1 Правила допустимости взаимодействий

Пусть $\mathcal{T}_{\text{allow}}$ — множество разрешённых пар типов компонентов, а $\tau(v)$ — тип вершины. Тогда
$$\forall (u, v) \in \mathcal{E} : \quad (\tau(u), \tau(v)) \in \mathcal{T}_{\text{allow}}. \tag{5}$$
Например, политика конкретного проекта может разрешать связи от балансировщика нагрузки к вычислительному сервису и от вычислительного сервиса к базе данных, запрещая клиенту прямое обращение к базе. Такое правило выражает *принятую архитектурную политику*; оно не является универсальным запретом для всех распределённых систем.

Допустимый путь в рассматриваемой политике
Клиент ⟶ Балансировщик ⟶ Сервис ⟶ База данных

Пример выявляемого нарушения
Клиент ⟶ База данных
Нарушена заданная матрица разрешённых типов взаимодействий.

Рис. 1: Пример структурной проверки по принятой политике связей

Структурная допустимость не гарантирует устойчивости по нагрузке. Даже ациклическая и связная топология может иметь узкое место в единственной базе данных или перегруженном сервисе. Именно поэтому следующий уровень исследования оценивает поведение запросов и очередей, а не только отношения между вершинами.

7 Проверяемые нефункциональные требования

Для детерминированной характеристики $m(\mathcal{G})$, например стоимости известного набора реплик, проверка порога θ имеет вид $m(\mathcal{G}) \le \theta$. Для случайной задержки, размера очереди и потерь необходимо явно задать окно наблюдения, статистический показатель и допустимую неопределённость оценки.

Ограничение стоимости можно записать как

$$C(\mathcal{G}) = \sum_{v \in \mathcal{V}_{\text{billable}}} r_v c_v^{\text{month}}, \tag{6}$$

где $\mathcal{V}_{\text{billable}}$ — оплачиваемые компоненты, r_v — число реплик, c_v^{month} — их заданная месячная стоимость. Модельные стоимости не должны выдаваться за реальные тарифы облачных провайдеров без источника и даты.

Соглашение об уровне обслуживания (Service Level Agreement, SLA) и целевой показатель его качества (Service Level Objective, SLO) не равны доказательству эксплуатационной надёжности. Даже статистически уверенный прогноз выполнения ограничения на 99-й перцентиль задержки относится лишь к заданной модели и нагрузке. Доступность нельзя вычислить по задержкам без модели отказов и восстановления.

8 Аналитическая проверка методами теории массового обслуживания

Нотация Кендалла M/M/1 означает пуассоновский поток поступающих запросов (экспоненциальные интервалы между ними), экспоненциальное обслуживание и один обслуживающий канал. В M/M/c количество одинаковых независимых каналов равно c. Предполагаются обслуживание в порядке прихода (First In, First Out, FIFO; первым пришёл — первым обслужен), бесконечная очередь и стационарный режим [2,16].

Пусть λ — интенсивность поступлений (запросов/с), μ — интенсивность обслуживания одного канала (запросов/с), а ρ — доля его загрузки. Для устойчивой M/M/1

$$\rho = \frac{\lambda}{\mu} < 1, \qquad W = \frac{1}{\mu - \lambda}, \qquad W_q = W - \frac{1}{\mu}, \tag{7}$$

где W — среднее время нахождения запроса в системе (ожидание и обслуживание), W_q — среднее время ожидания в очереди.

Расчётный пример. Для $\lambda = 10$ запросов/с и среднего обслуживания 50 мс имеем $\mu = 20$ запросов/с, $\rho = 0{,}5$, $W = 100$ мс и $W_q = 50$ мс. Это независимо проверяемые аналитические значения, а не результаты уже проведённого эксперимента.

Для M/M/c обозначим $a = \lambda/\mu$, $\rho = a/c < 1$. По формуле Эрланга C вероятность того, что запрос будет ожидать, равна

$$P_{\text{wait}} = \frac{\dfrac{a^c}{c!(1-\rho)}}{\displaystyle\sum_{n=0}^{c-1} \frac{a^n}{n!} + \frac{a^c}{c!(1-\rho)}}. \tag{8}$$

Среднее ожидание и среднее полное время ответа:

$$W_q = \frac{P_{\text{wait}}}{c\mu - \lambda}, \qquad W = W_q + \frac{1}{\mu}. \tag{9}$$

Конечные очереди, произвольные времена обслуживания, нестационарные пики и синхронные разветвления требуют других моделей. Приближённые формулы для очередей G/G/c могут помочь оценить среднее ожидание, но их нельзя напрямую трактовать как 95-й или 99-й перцентиль клиентской задержки.

9 Дискретно-событийная проверка архитектуры

В имитационной модели последовательно воспроизводятся события поступления запроса, постановки в очередь, начала и окончания обработки и переходов между сервисами. Виртуальное время моделирования не совпадает с длительностью выполнения программы. Для реализации эксперимента предполагается использовать SimPy: библиотеку событий, процессов и ограниченных ресурсов [5].

Для каждого компонента задаются число реплик, *отдельное* число обслуживающих каналов, дисциплина и длина очереди. Максимальная конкурентность не равна количеству каналов. В первом планируемом варианте входной поток пуассоновский, а длительность обслуживания случайна. При среднем времени $m > 0$ и стандартном отклонении s гамма-распределение задаётся параметрами

$$\alpha = \left(\frac{m}{s}\right)^2, \qquad \theta = \frac{s^2}{m}, \qquad s > 0. \tag{10}$$

Здесь α — параметр формы, θ — параметр масштаба; при $s = 0$ время обслуживания постоянно. Это *предположенная* форма распределения, а не следствие двух известных моментов.

Необходимо раздельно учитывать предложенную нагрузку, завершённые, отклонённые и незавершённые к границе окна заявки. Перцентиль времени ответа по *только завершённым* операциям не гарантирует выполнение SLO, если часть запросов была потеряна. Выводы «выполнено» и «нарушено» требуют независимых повторений моделирования с разными случайными начальными состояниями, прогревом и зафиксированным окном измерения. Статистическая неопределённость оценивается по независимым повторениям, а не путём ошибочного принятия всех событий одного прогона за независимые эксперименты.

10 Как особенности микросервисной топологии влияют на нагрузку

10.1 Альтернативные маршруты и параллельные обращения

Для получения оценок производительности недостаточно подсчитать количество узлов и связей. Необходимо определить, как *клиентские запросы превращаются в обращения к отдельным сервисам*. В исследуемом классе архитектур различаются два вида синхронной маршрутизации:

1. **Альтернативный маршрут**: из нескольких путей каждый запрос выбирает ровно один. При равномерном выборе k вариантов и входном потоке λ каждый получает ожидаемую интенсивность λ/k.

2. **Разветвление с несколькими обращениями** (fan-out): каждый входной запрос вызывает все k необходимых зависимостей. При отсутствии отказов и дополнительных условий каждый зависимый сервис получает интенсивность λ.

Пример. При $\lambda = 100$ запросов/с два альтернативных хранилища получают по 50 запросов/с. Если же сервис обращается к обоим хранилищам при каждом запросе, оба получают по 100 запросов/с. Равенство количества исходящих рёбер не делает эти архитектуры эквивалентными по нагрузке. Это различие заложено в исследуемой аналитической модели потоков AutoSystemDesign; симуляция альтернативного выбора реализуется отдельно, а полный синхронный fan-out с ожиданием всех ветвей остаётся этапом дальнейшей работы.

Для ациклического графа без повторных попыток интенсивность обращений к узлу v в общем случае можно выразить через ожидаемое число переходов по рёбрам:

$$\lambda_v = \lambda_v^{(0)} + \sum_{(u,v)\in\mathcal{E}} m_{uv}\lambda_u, \tag{11}$$

где $\lambda_v^{(0)}$ — внешний поток запросов непосредственно в v, а m_{uv} — среднее число обращений к v, порождаемых одним обращением к u. Для одной из двух равновероятных альтернатив $m_{uv} = 0{,}5$; для обязательного вызова $m_{uv} = 1$. Формула задаёт *средний поток*, но не его распределение, корреляции или задержки совместного выполнения. При циклах, таймаутах и повторных запросах требуется другая постановка.

10.2 Вероятностная эффективность кэша

Кэширование также изменяет поток к удалённому хранилищу. Пусть $h \in [0, 1]$ — вероятность успешного попадания в кэш для одного запроса, а λ_{cache} — интенсивность обращений к кэшу. Если каждое неуспешное обращение порождает ровно один запрос к базе данных, то

$$\lambda_{\text{DB}} = (1 - h)\lambda_{\text{cache}}. \tag{12}$$

При $\lambda_{\text{cache}} = 100$ запросов/с и $h = 0{,}8$ ожидаемая нагрузка на базу составляет 20 запросов/с. Число 20 здесь — *расчётное ожидание*, а не наблюдённая частота в одном имитационном прогоне.

В стохастической модели решение о попадании принимается для отдельного события. Поэтому число обращений к базе на конечном интервале случайно, даже при фиксированном h. В архитектурном проектировании важно различать аналитическое ожидание потока и фактические реализации запросов, а также заранее согласовать семантику пути промаха: какой узел инициирует обращение к базе и какие сетевые задержки включаются в клиентское время. Пока семантики аналитического расчёта и отдельных этапов SimPy не полностью унифицированы, их количественные результаты нельзя механически сопоставлять как две оценки одной модели.

10.3 Почему параллельные пути нельзя оценивать суммой средних

Если клиент должен дождаться двух синхронных обращений с длительностями T_1 и T_2, задержка параллельного участка равна $T = \max(T_1, T_2)$. Она не равна сумме времен ветвей. Для её математического ожидания в общем случае верно неравенство

$$\mathbb{E}[\max(T_1, T_2)] \geq \max\big(\mathbb{E}[T_1], \mathbb{E}[T_2]\big). \tag{13}$$

Например, пусть каждая из двух ветвей в равновероятных состояниях выполняется за 0 или 100 мс, причём если одна выполняется за 0 мс, другая — за 100 мс. Тогда обе имеют среднюю задержку 50 мс, но клиент всегда ожидает 100 мс. Следовательно, *максимум средних задержек* недооценивает среднее завершение параллельной операции. Это отдельный объект планируемого сравнения аналитики и имитации.

11 Ограниченные очереди и проверяемые границы результатов

11.1 Отказ в приёме запроса и незавершённые операции

В вычислительном сервисе необходимо различать число одновременно выполняемых запросов c, допустимое количество ожидающих q и общий предел допущенных запросов k. При раздельном учёте обслуживания и ожидания максимальная вместимость реплики равна

$$B = \min\{k, c + q\}. \tag{14}$$

Запрос отклоняется при достижении предела k либо когда заняты все каналы и очередь достигла q. Отказ имеет иную семантику, чем длительное ожидание: запрос не был допущен в систему обслуживания. Теоретические исследования многоканальных очередей конечной вместимости специально рассматривают вероятность потерь и их связь с размером буфера [17].

Для фиксированного окна наблюдения исход каждого измеряемого клиентского обращения должен принадлежать ровно одной категории:

$$N_{\text{arr}} = N_{\text{done}} + N_{\text{reject}} + N_{\text{unfinished}}, \tag{15}$$

где N_{arr} — число поступивших измеряемых запросов, N_{done} — число завершённых, N_{reject} — число отказов в допуске, $N_{\text{unfinished}}$ — число операций, не завершившихся до границы окна. Последние нельзя считать ни успешно завершёнными, ни мгновенно отклонёнными.

Расчётный пример. Пусть два запроса одновременно достигли базы данных с одним обслуживающим каналом, нулевой очередью и временем обработки 20 мс. Один запрос может начать обработку, а другой будет отклонён. Если расширить очередь до одного места, оба запроса могут завершиться последовательно. Это изменение модели ресурса влияет одновременно на долю отказов и время ответа, что невозможно вывести из одного среднего времени обслуживания.

11.2 Разграничение «задержки вперёд» и полного ответа клиенту

Текущие этапы графовой симуляции AutoSystemDesign моделируют прохождение запросов *вперёд* по выбранным сервисам и межсервисным связям. Полное время ответа клиенту (round-trip time, время прохождения запроса и возвращения ответа) может включать дополнительные события передачи ответа и ожидания связанных ветвей. Пока эти события не определены, прямую длительность маршрута нельзя автоматически назвать временем ответа клиенту и использовать для подтверждения его перцентильного SLO. Такой отказ от неподкреплённого вывода является свойством корректной методики проверки, а не дефектом инструмента.

12 Методика планируемого экспериментального исследования

План экспериментов строится по принципу разделения целей исследования, способа создания исходных данных, оцениваемых величин, сравниваемых методов и итоговых показателей (схема ADEMP из методологии имитационных исследований) [18]. Отдельно фиксируются источники случайности и критерии остановки. Это предотвращает подгонку протокола под результаты, полученные на одной языковой модели.

12.1 Проверка оценщиков отдельно от языковой модели

Эксперимент A проводится на фиксированных архитектурах без вызова LLM. Для M/M/1 и M/M/c сопоставляются точные аналитические значения с результатами достаточно длительной имитации. Для более сложных систем заранее подготавливается независимый эталон: отдельная реализация симуляции с установленной семантикой либо нагрузочное тестирование развёрнутого прототипа. Проверять точность симулятора сравнением с самим собой нельзя.

Сценарии будут варьировать интенсивность и характер нагрузки, число обслуживающих каналов, ёмкость очереди, вариативность обслуживания, число реплик и способ маршрутизации. Обязательно включаются перегрузки, пиковые режимы, неизвестные параметры и неподдерживаемые типы взаимодействий. Помимо погрешностей численных оценок фиксируется доля ситуаций, в которых метод честно отказывается делать вывод.

12.2 Влияние проверок на генерацию LLM

Эксперимент B использует одинаковые текстовые требования и профили нагрузки для пяти сравниваемых способов проектирования:

1. Однократная генерация без обратной связи.
2. Однократная генерация с последующей проверкой, но без исправления.
3. Итеративная генерация с диагностикой нарушений структуры.
4. Итеративная генерация со структурной и аналитической проверкой ТМО.
5. Итеративная генерация со структурной, аналитической и имитационной проверками.

Внешняя проверка итоговой архитектуры должна быть одинаковой для всех вариантов, а её результат нельзя подменять самооценкой модели. Сравниваются как локально запускаемые LLM, так и другие доступные модели. Список точных версий, аппаратная конфигурация, температура, формат ответа, лимиты токенов и число итераций будут зафиксированы *до* основного исследования.

12.3 Показатели и статистическая процедура

Главным показателем безопасности проверки выбрана частота *ложного подтверждения выполнения требований*. Пусть N_{viol} — количество сценариев с независимо установленными нарушениями, а $N_{\text{false pass}}$ — число сценариев, ошибочно признанных удовлетворяющими ограничениям. Тогда

$$R_{\text{false pass}} = \frac{N_{\text{false pass}}}{N_{\text{viol}}}, \qquad N_{\text{viol}} > 0. \tag{16}$$

Дополнительно измеряются доля структурно допустимых графов, ошибки среднего и перцентильного времени ответа, частота отказов, пропускная способность, доля оценённых сценариев, число итераций, расход токенов и время расчёта. Если метод возвращает «не оценивалось», такой случай нельзя исключать из отчёта о покрытии.

Для языковых моделей предусматриваются независимые повторы генерации; для симулятора — независимые репликации с контролируемыми случайными начальными состояниями. Различия между сравниваемыми вариантами будут оцениваться с учётом парности сценариев и доверительных интервалов. Объём выборки, первичные критерии и правило обработки множественных сравнений подлежат фиксации до изучения итогов.

13 Состояние работы и ограничения

В проекте AutoSystemDesign уже имеются средства представления и структурной проверки архитектурных графов. Аналитическая проверка на основе TMO и моделирование SimPy разрабатываются в отдельных pull request и не считаются полностью интегрированными и экспериментально принятыми. В статье намеренно разделены формальная постановка, существующие компоненты стенда и *запланированные* численные опыты.

Основные угрозы достоверности: различие между принятым семейством распределений и реальным трафиком, неопределённость маршрутизации, неполная модель отказов, погрешность эталона и возможное попадание тестовых требований в обучающие данные LLM. До проведения запланированных опытов выводы о статистически значимом превосходстве какого-либо метода или модели делать нельзя.

14 Заключение

Сформулирована задача сравнительной проверки микросервисных архитектур, создаваемых большими языковыми моделями, с разграничением структурных ограничений, аналитических оценок очередей и дискретно-событийного моделирования. Показано на уровне математической постановки, что среднее время ожидания в стационарной модели M/M/1 или M/M/c не является универсальным свидетельством выполнения перцентильных требований при конечных очередях, перегрузках и сложной маршрутизации.

Предложена программа двух взаимосвязанных исследований: независимая проверка численных оценщиков на контрольных архитектурах и сравнение способов генерации с различной диагностической обратной связью. Основными ожидаемыми результатами будущих экспериментов являются количественные оценки риска ложного подтверждения соответствия, покрытия проверок, точности задержек, вычислительных расходов и чувствительности разных LLM к обратной связи.

На текущем этапе экспериментальных результатов, подтверждающих выдвинутые гипотезы, нет. Исследовательская ценность последующей работы будет определяться воспроизводимостью сценариев, независимостью эталонов и статистически обоснованным сравнением методов, а не самим фактом реализации программного комплекса.

Список литературы

[1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 3rd ed., 2015.

[2] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[3] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[4] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," *arXiv preprint arXiv:2409.02977*, 2024.

[5] "Simpy: Discrete event simulation for python." https://simpy.readthedocs.io/en/stable/. Official documentation, accessed October 22, 2026.

[6] S. Becker, H. Koziolek, and R. H. Reussner, "The palladio component model for model-driven performance prediction," *Journal of Systems and Software*, vol. 82, no. 1, pp. 3–22, 2009.

[7] M. Esposito, X. Li, S. Moreschini, N. Ahmad, T. Cerny, K. Vaidhyanathan, V. Lenarduzzi, and D. Taibi, "Generative ai for software architecture: Applications, challenges, and future directions," *Journal of Systems and Software*, vol. 231, p. 112607, 2026.

[8] L. Schmid, T. Hey, M. Armbruster, S. Corallo, D. Fuchss, J. Keim, H. Liu, and A. Koziolek, "Software architecture meets llms: A systematic literature review," 2025.

[9] A. Ferrari, S. Abualhaija, and C. Arora, "Model generation with llms: From requirements to uml sequence diagrams," in *Proceedings of the 32nd IEEE International Requirements Engineering Conference Workshops (REW 2024)*, pp. 291–300, 2024.

[10] P. Giannouris and S. Ananiadou, "Nomad: A multi-agent llm system for uml class diagram generation from natural language requirements," in *Proceedings of the 14th International Conference on Model-Based Software and Systems Engineering (MODELSWARD)*, pp. 257–264, SciTePress, 2026.

[11] R. Li, Y. Zhang, X. Zhou, P. Liang, W. Sun, J. Xuan, Z. Jin, and Y. Liu, "Maad: Automate software architecture design through knowledge-driven multi-agent collaboration," *arXiv preprint arXiv:2507.21382*, 2025.

[12] K. Szczepanik and J. A. Chudziak, "Collaborative llm agents for c4 software architecture design automation," *arXiv preprint arXiv:2510.22787*, 2025.

[13] M. Gokan Khan, J. Taheri, A. Al-Dulaimy, and A. Kassler, "Perfsim: A performance simulator for cloud native microservice chains," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1395–1413, 2023.

[14] J. Wu, M. Xu, Y. He, K. Ye, and C. Xu, "Cloudnativesim: A toolkit for modeling and simulation of cloud-native applications," *arXiv preprint arXiv:2409.05093*, 2024.

[15] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, C. Hu, B. Ritchken, B. Jackson, *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in *Proceedings of the 24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 3–18, 2019.

[16] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. John Wiley & Sons, 1975.

[17] D. W. Choi, N. K. Kim, and K. C. Chae, "A two-moment approximation for the gi/g/c queue with finite capacity," *INFORMS Journal on Computing*, vol. 17, no. 1, pp. 75–81, 2005.

[18] T. P. Morris, I. R. White, and M. J. Crowther, "Using simulation studies to evaluate statistical methods," *Statistics in Medicine*, vol. 38, no. 11, pp. 2074–2102, 2019.